

Version 1. Integrate External API with Local Database

Create a Flask app that:

1. Fetches user data from [JSONPlaceholder API](#).
2. Stores the fetched users into a local SQLite database.
3. Provides `/users` endpoint to return all users from the database in JSON format.

Practice integrating external API + local DB.

Version 2. CSV File Integration with Flask API

1. Create a Flask route `/upload` to upload a CSV file (e.g., `students.csv`).
2. Parse the CSV file using `pandas` or `csv` module.
3. Store the records into a SQLite table named `students`.
4. Create `/students` route to show all data as JSON.

Integrate file input → database → REST API output.

Version 3. Weather + City Data Integration\

1. Create a CSV file `cities.csv` with columns: `city`, `population`.
2. Use Flask to fetch weather data from the OpenWeatherMap API for each city.
3. Combine local CSV data with API data and return as JSON at `/city-weather`.

Integrate local + external API and serve combined results.

Version 4. Integrate Two Flask Services

Create two small Flask apps:

- User Service: `/users` returns user info.
- Order Service: `/orders` returns order info.

Then, in a main Flask app, create `/user-orders` route that:

- Calls both microservices (using `requests` module).
- Merges users and orders into one JSON response.

Integrate multiple internal REST services.

Version 5. Integration + Visualization Dashboard

1. Build a Flask backend that merges:
 - `sales.csv` file (local data)
 - Currency rates from API ([exchangerate.host](#))
2. Convert all sales prices to USD using live exchange rates.
3. Display the results on a webpage using an HTML template (Jinja2).

Integrate data and show it in a simple web dashboard.